

# Merge Pattern

## Core Concepts

Before solving problems, be comfortable with:

- Merge two sorted linked lists
- Merge multiple sorted linked lists
- Maintain sorted order while merging
- Dummy node technique
- Tail pointer
- Divide & Conquer for merging K lists
- Merge Sort on Linked Lists

**Time Complexity:** Usually  $O(n)$

**Space Complexity:**  $O(1)$  (for two-list merge)

## LeetCode 21 – Merge Two Sorted Lists\*\*\*\*\*

### Concept

- Use a **dummy node** to avoid handling the head separately.
- Maintain a **tail** pointer to the last node of the merged list.
- Compare the current nodes of both lists, attach the smaller one, and move its pointer.
- When one list ends, attach the remaining nodes of the other list.

**Time:**  $O(n + m)$

**Space:**  $O(1)$

### Edge Cases

- One list is empty.
- Both lists are empty.
- Duplicate values.
- All nodes of one list are smaller than the other.

### Concept Code Snippet

```
ListNode dummy(-1);
ListNode* tail = &dummy;

while (list1 && list2) {
    if (list1->val <= list2->val) {
        tail->next = list1;
        list1 = list1->next;
    } else {
        tail->next = list2;
        list2 = list2->next;
    }
    tail = tail->next;
}

// Attach remaining nodes
tail->next = (list1) ? list1 : list2;

return dummy.next;
```

**Pattern to Remember:**

Dummy Node + Tail Pointer + Compare + Attach Remaining

## LeetCode 1669 – Merge In Between Linked Lists\*\*\*\*\*

### Concept

- Find the node **just before index a** (`prevA`).
- Find the node **just after index b** (`nextB`).

- Connect `prevA` to the head of `list2`.
- Traverse to the tail of `list2` and connect it to `nextB`.

Time: `O(n + m)`

Space: `O(1)`

### Edge Cases

- `a = 0` (if allowed, use a dummy node).
- `b` is the last index of `list1`.
- `list2` has only one node.
- Entire middle portion is removed.

### Concept Code Snippet

```
// Find node before 'a'
ListNode* prevA = list1;
for (int i = 0; i < a - 1; i++)
    prevA = prevA->next;

// Find node after 'b'
ListNode* nextB = prevA;
for (int i = a - 1; i <= b; i++)
    nextB = nextB->next;

// Connect list2
prevA->next = list2;

// Find tail of list2
ListNode* tail = list2;
while (tail->next)
    tail = tail->next;

// Connect remaining list1
tail->next = nextB;
```

### Pattern to Remember:

Find `prevA` → Find `nextB` → Connect `list2` → Connect `tail(list2)` to `nextB`

## LeetCode 148 – Sort List ★★★★★ \*\*\*\*\*

### Concept

- Use **Merge Sort** since linked lists don't support efficient random access.
- Find the middle using **slow & fast pointers**.
- Split the list into two halves.
- Recursively sort both halves.
- Merge the two sorted lists.

Time: `O(n log n)`

Space: `O(log n)` (recursive stack)

### Edge Cases

- Empty list.
- Single node.
- Already sorted list.
- Duplicate values.

### Concept Code Snippet

#### 1. Find Middle & Split

```
ListNode *slow = head, *fast = head->next;

while (fast && fast->next) {
    slow = slow->next;
    fast = fast->next->next;
}

ListNode* second = slow->next;
slow->next = nullptr;    // Split into two halves
```

2. Recursion Merge Sort

```
if (!head || !head->next)
    return head;

ListNode* left = sortList(head);
ListNode* right = sortList(second);

return merge(left, right);
```

3. Merge Two Sorted Lists

```
ListNode dummy(-1);
ListNode* tail = &dummy;

while (left && right) {
    if (left->val <= right->val) {
        tail->next = left;
        left = left->next;
    } else {
        tail->next = right;
        right = right->next;
    }
    tail = tail->next;
}

tail->next = left ? left : right;
return dummy.next;
```

Pattern to Remember

Find Middle → Split → Recursively Sort Both Halves → Merge

LeetCode 328 – Odd Even Linked List

\*\*\*\*\*

Concept

- Maintain two pointers:
  - **odd** → nodes at odd indices.
  - **even** → nodes at even indices.
- Save the head of the even list.
- Rearrange pointers to separate odd and even nodes.
- Attach the even list after the odd list.

Time: **O(n)**

Space: **O(1)**

Edge Cases

- Empty list.
- Single node.
- Two nodes.
- Odd/even number of nodes.

Concept Code Snippet

```
ListNode* odd = head;
ListNode* even = head->next;
ListNode* evenHead = even;

while (even && even->next) {
    odd->next = even->next;
    odd = odd->next;

    even->next = odd->next;
    even = even->next;
}

// Append even list
odd->next = evenHead;

return head;
```

Pattern to Remember

Odd Pointer + Even Pointer + Save Even Head → Separate → Attach Even List at End

\*\*\*\*\* IMPORTANT\*\*\*\*\*  
\*\*\*\*\* IMPORTANT\*\*\*\*\*

LeetCode 23 – Merge K Sorted Lists ★★ ★★ ★★

brute force : put in array all the elements and sort

another : iterate over the heads of all the lists and pick the min and add to the answer

Concept

There are two common approaches:

1. Min Heap (Most Common)

- Push the head of every non-empty list into a **min heap**.
- Repeatedly pop the smallest node, attach it to the answer.
- If that node has a next node, push it into the heap.

Time: **O(N log K)**

Space: **O(K)**

2. Divide & Conquer

- Repeatedly merge pairs of lists (like Merge Sort).
- Continue until only one list remains.

Time: **O(N log K)**

Space: **O(log K)** (recursive) / **O(1)** iterative

**N** = Total nodes, **K** = Number of lists.

Edge Cases

- Empty **lists** vector.
- Some lists are empty.
- Only one list.
- Duplicate values.

Concept Code Snippet (Min Heap)

```

auto cmp = [](ListNode* a, ListNode* b) {
    return a->val > b->val;
};

priority_queue<ListNode*, vector<ListNode*>, decltype(cmp)> pq(cmp);

for (auto node : lists)
    if (node) pq.push(node);

ListNode dummy(-1);
ListNode* tail = &dummy;

while (!pq.empty()) {
    ListNode* node = pq.top();
    pq.pop();

    tail->next = node;
    tail = tail->next;

    if (node->next)
        pq.push(node->next);
}

return dummy.next;

```

### Concept Code Snippet (Divide & Conquer)

```

ListNode* mergeK(vector<ListNode*>& lists, int l, int r) {
    if (l == r) return lists[l];

    int mid = l + (r - l) / 2;

    ListNode* left = mergeK(lists, l, mid);
    ListNode* right = mergeK(lists, mid + 1, r);

    return merge(left, right);    // LC 21
}

```

### Pattern to Remember

- Min Heap: Push K Heads → Pop Smallest → Push Next → Repeat
- Divide & Conquer: Split Lists → Merge Pairs → Repeat Until One List

MIN HEAP ANSWER :

TC =  $K \log K + N \log K$

SC =  $1 + K$

```

ListNode* mergeKLists(vector<ListNode*>& lists) {
    if(lists.size() == 0) return NULL;

    ListNode* dummyHead = new ListNode(-1);
    ListNode* dummyTail = dummyHead;

    priority_queue<pair<int, ListNode*>, vector<pair<int, ListNode*>>, greater<pair<int, ListNode*>>> pq;
    for(auto head : lists) if(head != NULL) pq.push({head->val, head});

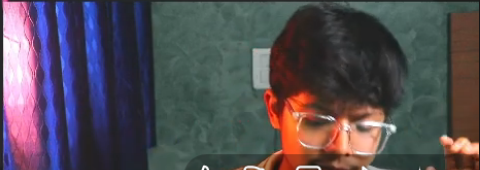
    while(!pq.empty()){
        ListNode* minNode = pq.top().second;
        pq.pop();
        if(minNode->next != NULL) pq.push({minNode->next->val, minNode->next});

        dummyTail->next = minNode;
        dummyTail = dummyTail->next;
    }
    return dummyHead->next;
}

```

*Handwritten notes:*

- Node* (with a circle around 'Node')
- val* → (with an arrow pointing to the first element of a pair in the code)
- Min Heap* (with an arrow pointing to the priority\_queue declaration)
- Pushing Heads of K Lists in Heap* (with an arrow pointing to the for loop pushing heads into the heap)
- Grab Min-value Node* (with an arrow pointing to the pq.top().second line)
- Push min-value node in Heap* (with an arrow pointing to the pq.push line for the next node)



ANOTHER METHOD :

combine two lists at a time until all are combined

TC =  $N * K$

SC = 1

DIVIDE CONQUER ONE :

TC =  $N \log K$

SC = 1

```
public:
ListNode* mergeKLists(vector<ListNode*>& lists) {
    if(lists.size() == 0) return NULL;
    return mergeKListsHelper(lists, 0, lists.size()-1);
}

ListNode* mergeKListsHelper(vector<ListNode*>& lists, int start, int end) {
    if(start > end) return NULL;
    if(start == end) return lists[start];

    int mid = start + (end-start)/2;
    ListNode* left = mergeKListsHelper(lists, start, mid);
    ListNode* right = mergeKListsHelper(lists, mid + 1, end);
    return merge(left, right);

    ListNode* merge(ListNode* list1Head, ListNode* list2Head) {
        ListNode* dummyHead = new ListNode(-1);
        ListNode* dummyTail = dummyHead;
    }
}
```

base case

base case of recursion

split lists into 2 halves

Merge 2 halves

